Inteligência Artificial

Introdução ao Aprendizado de Máquina Clássico – Parte II

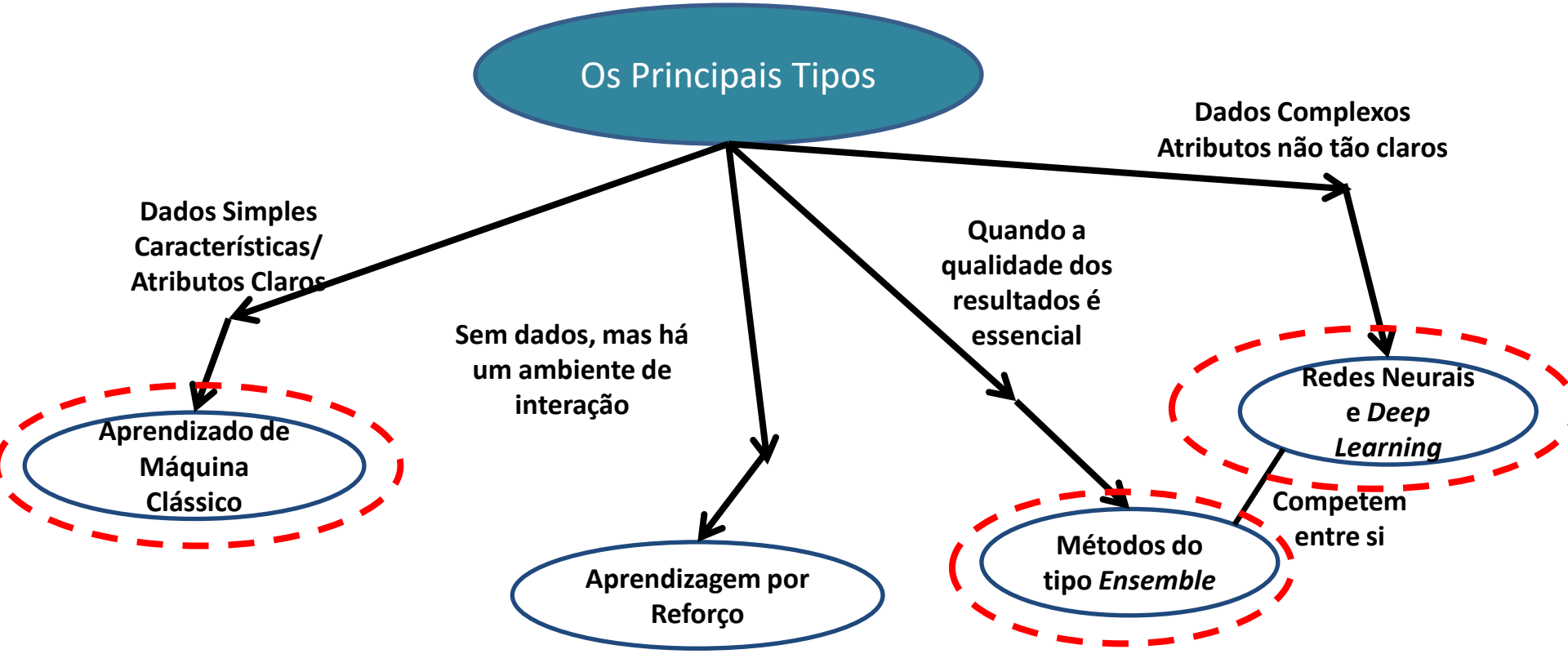
Classificação –Dilema Viés-Variância, Regressão Logística e KNN, Avaliação de Modelos Preditivos e scikitlearn

Prof. Dr. Ivan Carlos Alcântara de Oliveira

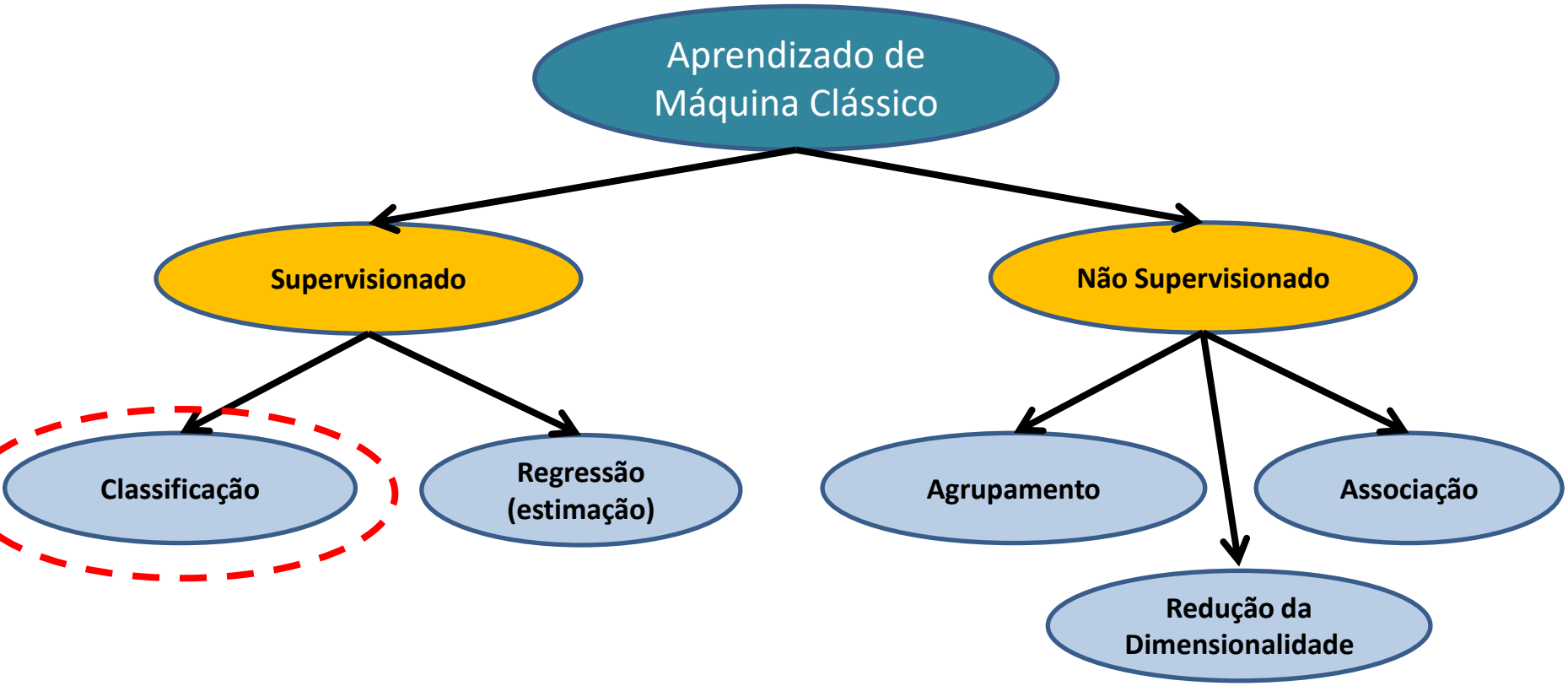
Baseado no material do prof. Dr. Rogério de Oliveira e prof. Dr. Ivan Carlos Alcântara de Oliveira

https://orcid.org/0000-0002-6020-7535

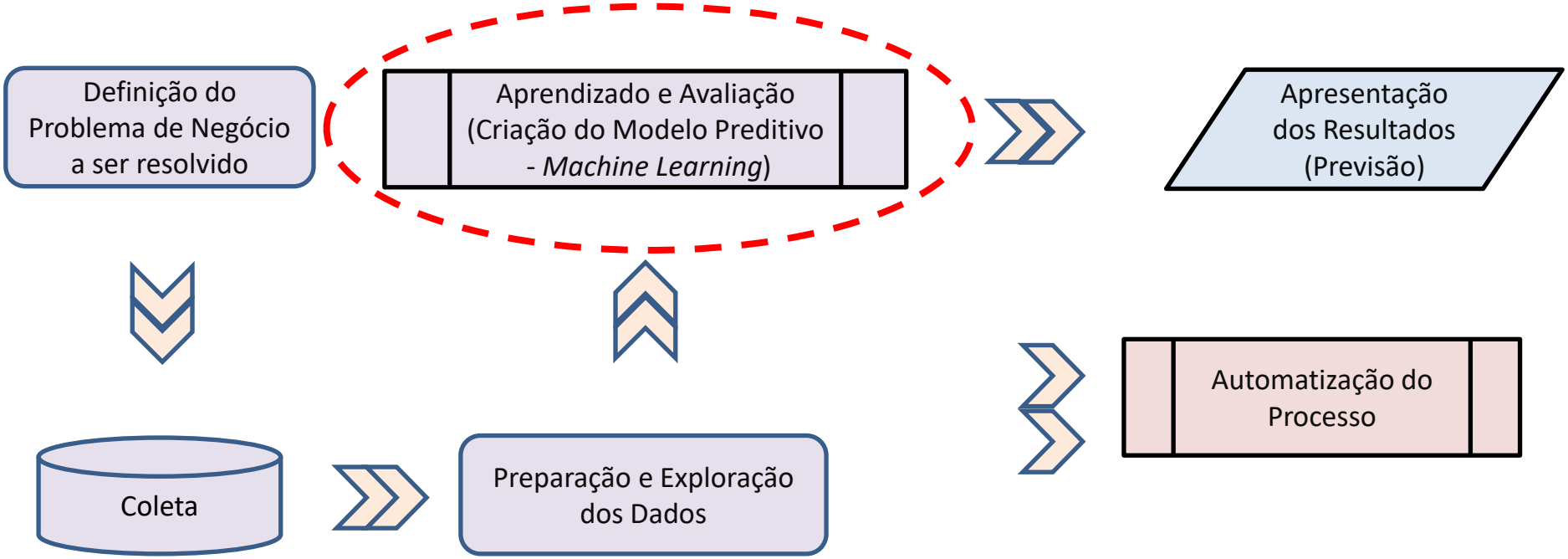
Os principais tipos de Aprendizado de Máquina



Aprendizado de Máquina Clássico



Processo de Análise de Dados



Modelagem: Aprendizado Supervisionado

Elementos do processo de Aprendizagem

Entrada

X { Atributos Preditores }

Saída

Y { Atributo Alvo }

Função real

f: X -> Y { Relaciona os atributos preditores ao atributo alvo }

Dados

(x1, y1), (x2, y2), ... (xn, yn)

{ Dados de histórico necessários para treinar o modelo }

{ Para fazer previsões para o futuro, observa-se o passado, ou seja, busca-se os padrões do passado para prever o futuro }

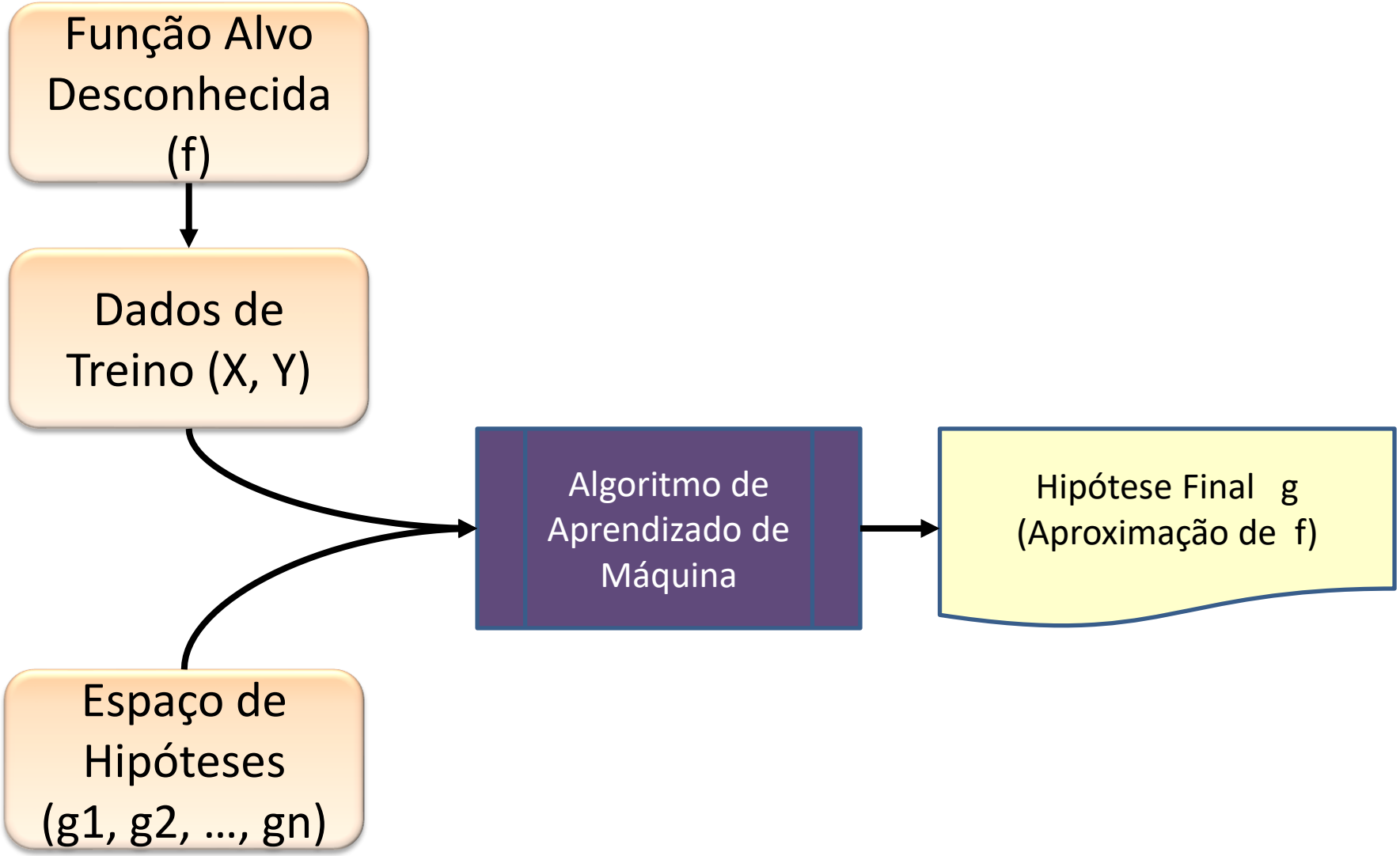
Hipótese

g: X -> Y

{ Aproximação da função f , ou seja, função a ser descoberta pelo algoritmo de Aprendizado de Máquina. Há um espaço de hipóteses e o algoritmo, a partir de pontuações atribuídas a cada uma delas, determina a melhor hipótese (aproximação da função f) }

Modelagem: Aprendizado Supervisionado

Elementos do processo - Síntese

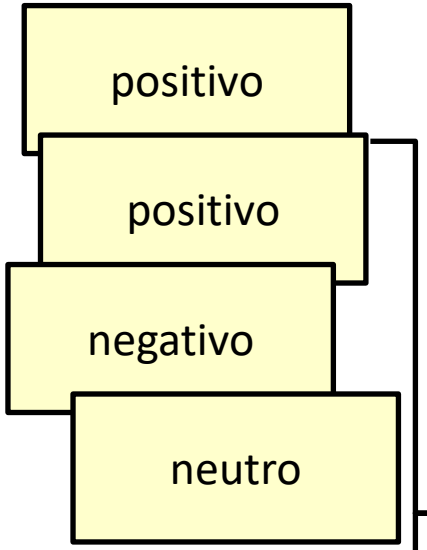


Classificação

Análise de Sentimentos em Redes Sociais

Atributos de Entrada

- Posts de usuários em rede social



Atributo Alvo (Rótulo ou Saída) (**Sentimento – Positivo, negativo ou neutro**)

Algoritmo de Aprendizado de Máquina Supervisionado (Classificação)

Modelo Preditivo F(x)

Novo post de entrada (Sentimento ???)

Previsão: Positivo, Negativo ou Neutro

Dataset

Dados de Histórico (Amostra significativa- conjunto de exemplos)

Aprendizado de Máquina

Conceitos – Indução de Hipóteses

- **Induzida uma hipótese (função – modelo preditivo)**, é **desejável** que ela seja **válida** para **outros objetos** do **mesmo domínio ou problema** que **não fazem parte do conjunto de treinamento**.

Importante!

- A essa capacidade de uma hipótese continuar a ser válida para novos objetos dá-se o nome de **capacidade de generalização da hipótese (modelo preditivo)**.

Aprendizado Supervisionado

Algoritmo – Dilema Viés- Variância

- Parece bastante simples construir esses modelos, mas **construir bons modelos pode ser uma tarefa bastante árdua**.

- Vamos nos deter um pouco mais sobre **como podemos avaliar os modelos**.

- Vamos começar entendo o que é conhecido como ****Dilema Viés-Variância**** (ou **Bias−variance tradeoff**).

Aprendizado Supervisionado

Algoritmo – Dilema Viés- Variância - *underfitting*

- A **capacidade de um modelo de capturar a verdadeira relação entre as variáveis preditoras e a variável alvo** é chamado **viés (****bias****)**.
- Um **alto erro de viés** significa que o **modelo não se ajusta aos dados** e que, portanto, **não consegue representar os dados**.
- No limite ele **não está aprendendo** nada.
- É o que chamamos de **subajuste do modelo**, ou ***underfitting***.

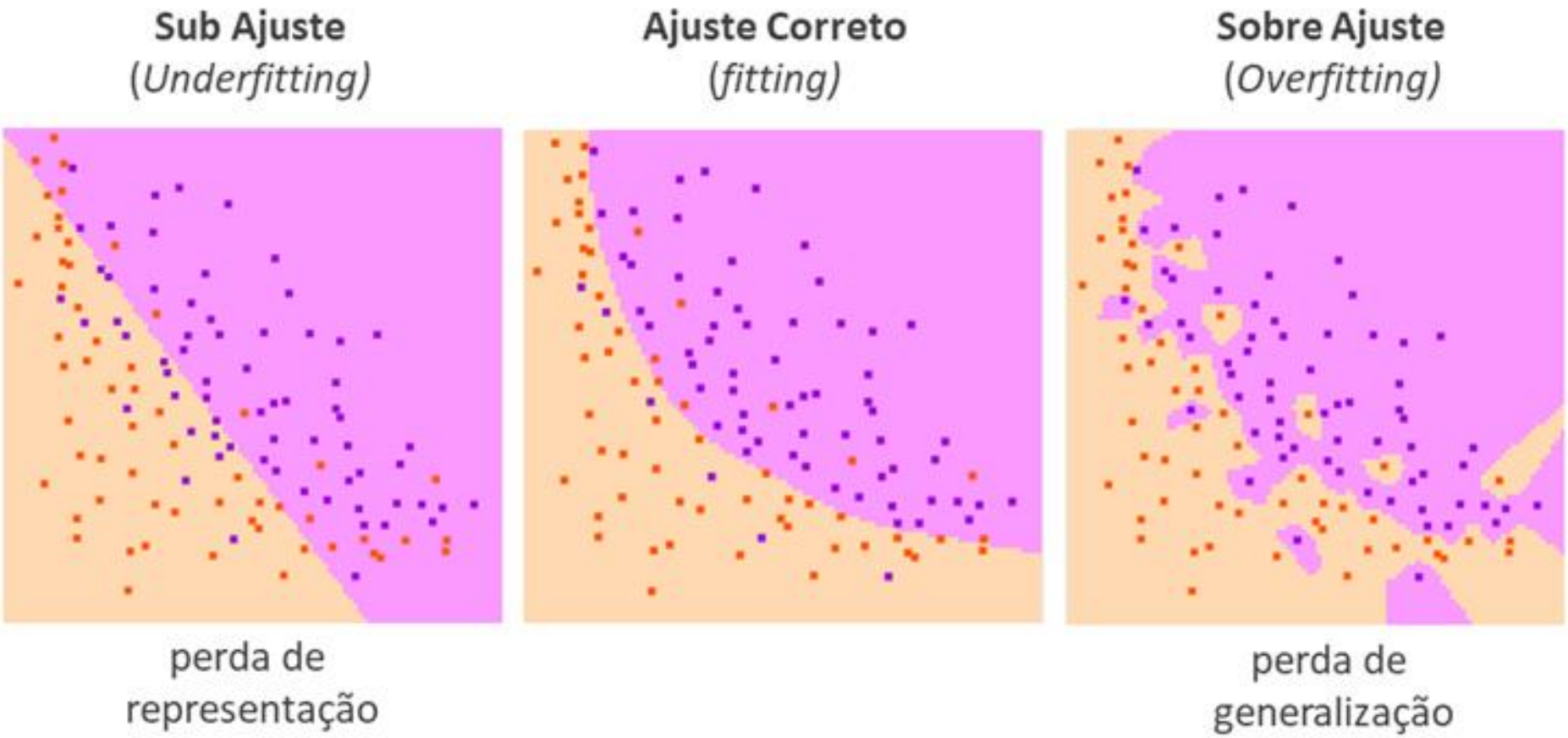
Aprendizado Supervisionado

Algoritmo – Dilema Viés- Variância - *overfitting*

- Entretanto, **ao tentarmos reduzir o erro de viés** dos dados **podemos nos deparar** com um **outro problema**.
- Com **um viés muito pequeno, o modelo poder ficar tão ajustado aos dados de treinamento que falha ao tentar prever novos casos**.
- Neste caso, **o modelo captura toda a variância dos dados**.
- O modelo **erra ao *generalizar* novos casos**. Isso é o que chamamos de ****subreajuste do modelo****, ou ****overfitting***.

Aprendizado Supervisionado

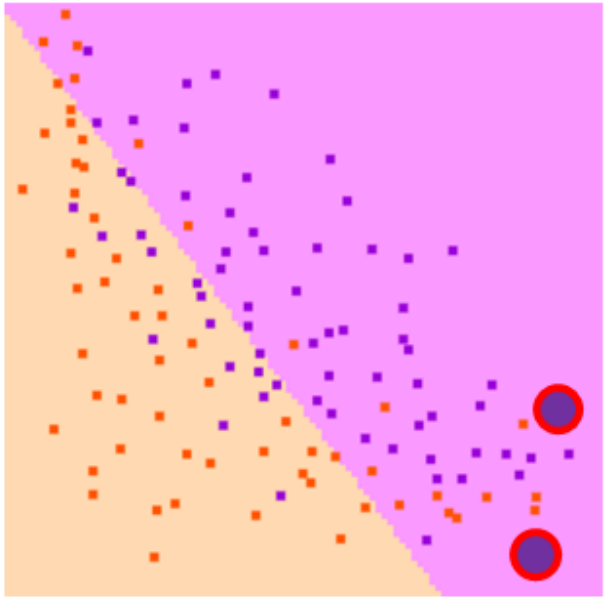
Algoritmo – Dilema Viés- Variância



Aprendizado Supervisionado

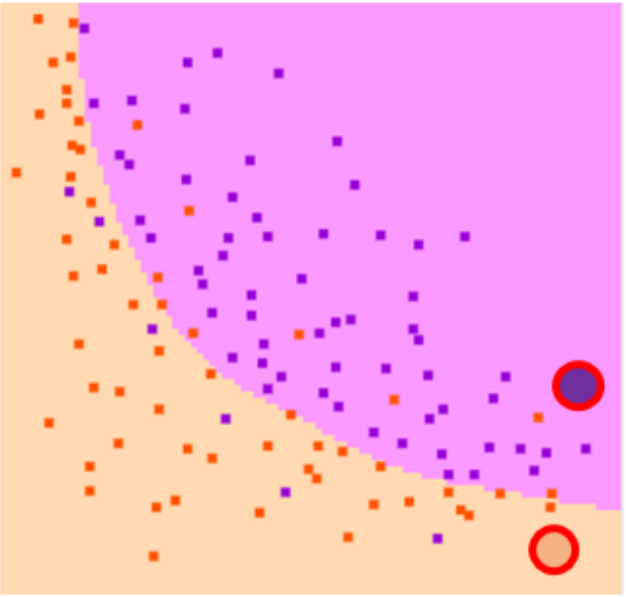
Algoritmo – Dilema Viés- Variância

Sub Ajuste
(Underfitting)



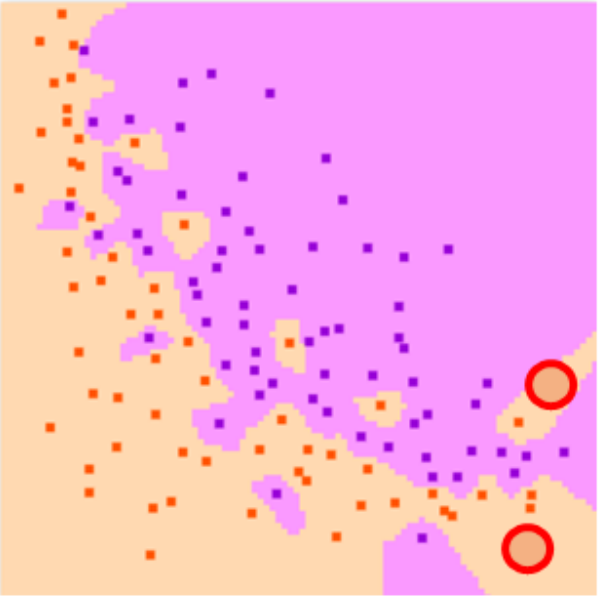
Erro de Classificação

Ajuste Correto
(fitting)



Classificação Mais Provável

Sobre Ajuste
(Overfitting)



Erro de Classificação

Aprendizado Supervisionado

Algoritmo – Diminuindo a possibilidade de *overfitting* – Conjuntos de Treinamento e Teste

- **Aprender ou treinar parâmetros de um modelo de predição e testá-lo com os mesmos dados é um erro de método**.

- É como **dar uma prova para avaliar o conhecimento de um aluno somente com exercícios que já foram dados previamente na sala da aula**.

- Seria um **modelo sem grande utilidade para predição de novos casos** (*overfitting*).

Aprendizado Supervisionado

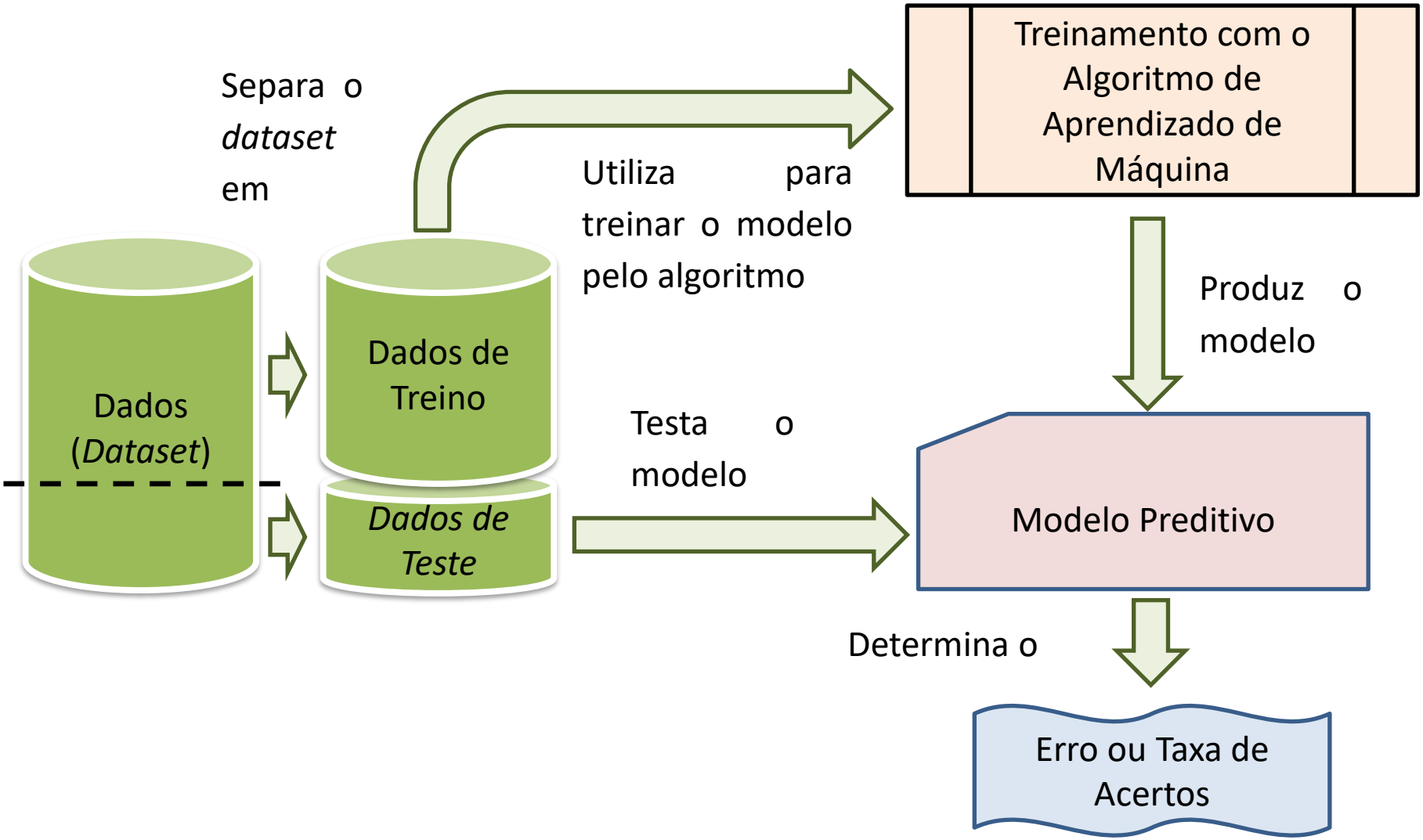
Algoritmo – Diminuindo a possibilidade de *overfitting* – Conjuntos de Treinamento e Teste

- Para diminuir a chance de *overfitting*, temos que **definir dois conjuntos diferentes de dados**, **um conjunto de Treinamento** e **um conjunto de Teste**:
  - O conjunto de treinamento será **empregado para o treinamento dos parâmetros do modelo;**
  - O conjunto de teste **será empregado para avaliar o modelo preditivo ajustado.**

- **O dilema está, portanto, em obter o ponto de equilíbrio entre o subajuste (*underfitting*) e sobreajuste (*overfitting*) dos dados**.

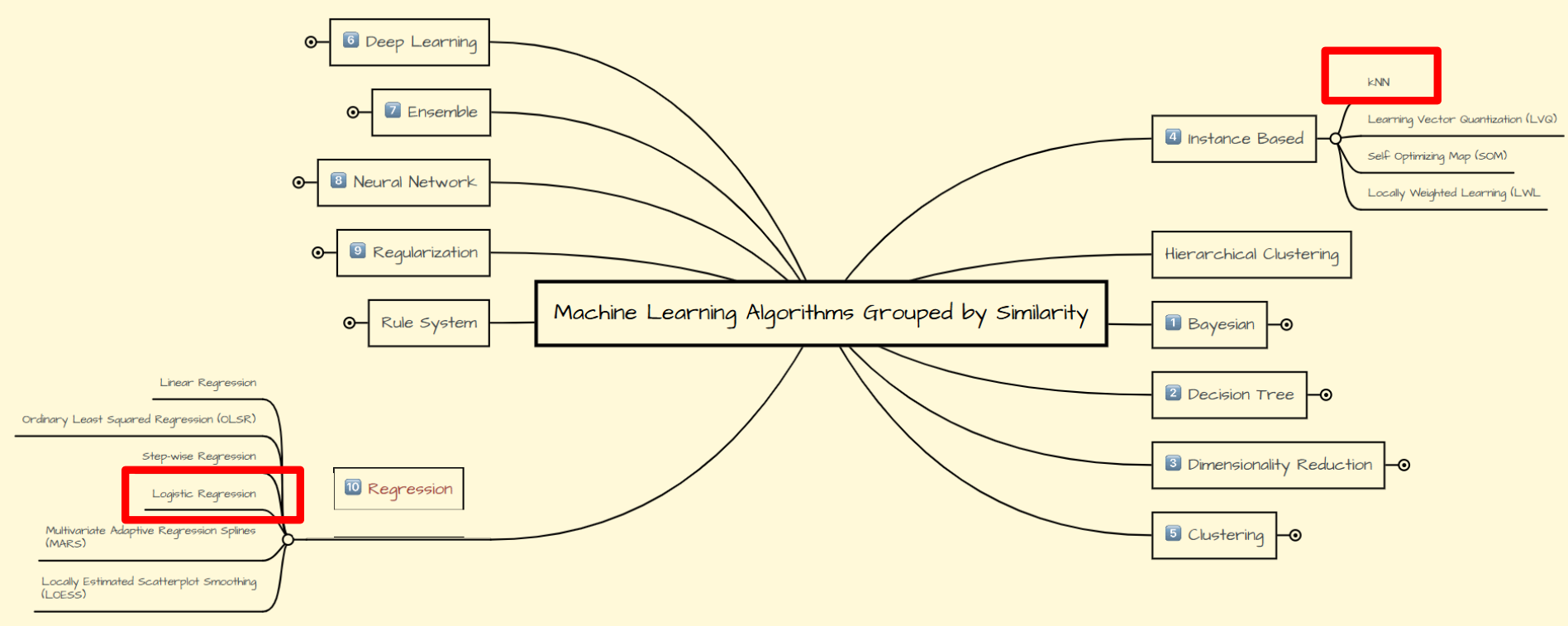
Modelagem: Aprendizado Supervisionado

O processo de Aprendizagem – Treinamento e Teste



Aprendizado de Máquina

Algoritmos de Machine Learning Agrupados por Similaridade



Fonte: https://www.mindmeister.com/pt/675097196/machine-learning-algorithms-grouped-by-similarity?fullscreen=1#. Data da consulta: 07/03/2026.

Aprendizado Supervisionado

Algoritmo – Regressão Logística

- É um **algoritmo de classificação** do aprendizado supervisionado de máquina, dentro da **categoria de modelos de regressão**.

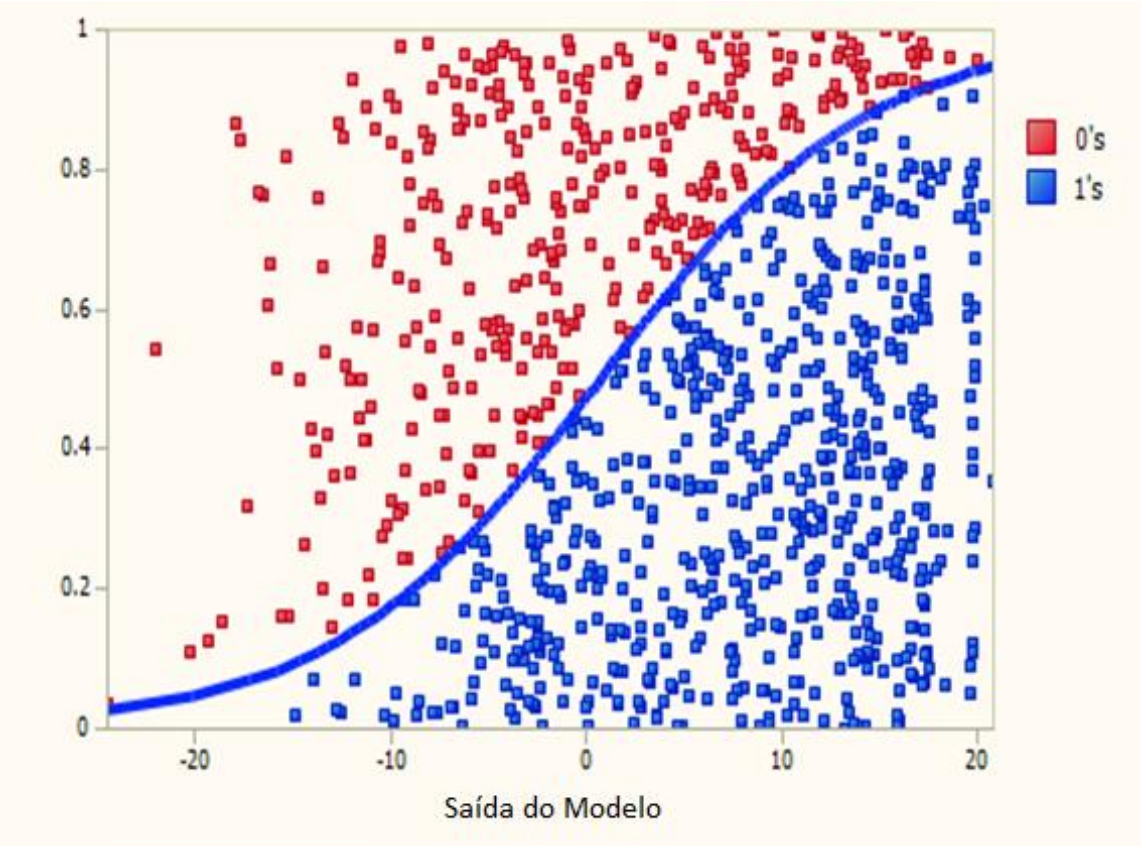
- Também é amplamente **utilizado em algumas camadas de redes neurais artificiais**.

- O que se **deseja** com o algoritmo é **encontrar uma linha (função) que separa os dados das classes**.

Aprendizado Supervisionado

Algoritmo – Regressão Logística

- O **objetivo é prever uma classe**, como apresentado no gráfico ao lado.
- Observe a linha divisória.



- Considerando um novo dado, o modelo verifica se ele pertence a categoria azul ou vermelha.

Aprendizado Supervisionado

Algoritmo – Regressão Logística

- A Regressão Logística **modela as probabilidades para problemas de classificação binários**.
- Um nome **melhor seria Classificador Logístico**, no lugar de Regressão Logística, evitando confusão com regressão.
- **A saída do modelo é uma única probabilidade P**.
- Por exemplo, as probabilidades a serem calculadas são:

$$P(Y=Benigno)$$

$$P(Y \neq Benigno) = 1 - P(Y=Benigno)$$

Aprendizado Supervisionado

Algoritmo – Regressão Logística

- A **regressão logística modela** o que é conhecido como **log de probabilidades**:

$$\mathbf{log}\left(\frac{p}{1-p}\right) = a_0 + a_1x_1 + a_2x_2 + \ldots + \mathbf{anxn}$$

- O lado esquerdo é chamado "odds logarítmicas" (chances logarítmicas) ou "logit“.
- Para encontrar a **probabilidade p (chance de ocorrer)**, basta aplicar a função exponencial dos dois lados e isolar p obtendo-se:

Aprendizado Supervisionado

Algoritmo – Regressão Logística

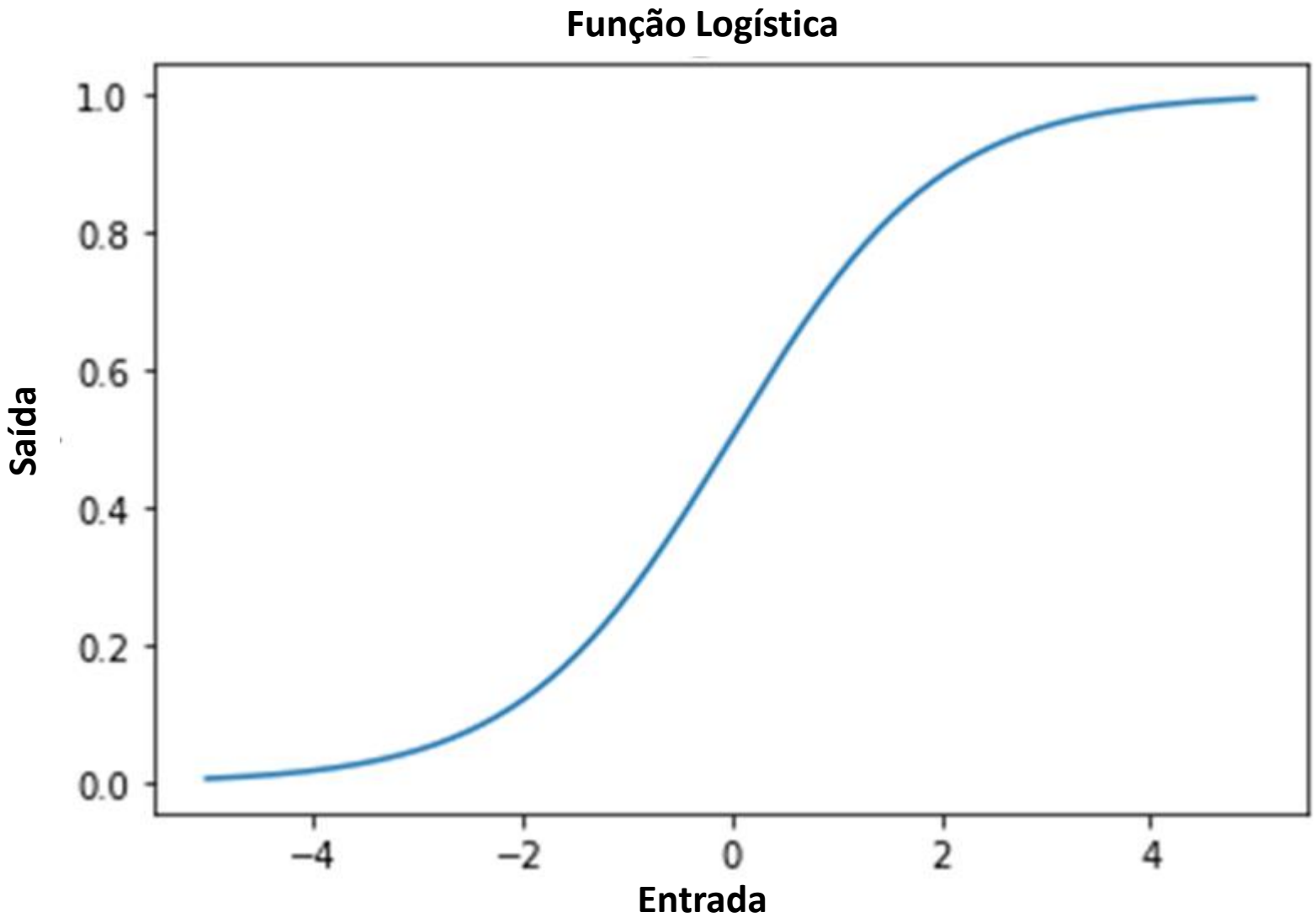
$$\mathbf{p} = {}^{1}\Big/_{\left(1+e^{-(a_0+a_1x_1+a_2x_2+\ \ldots+anxn)}\right)} =$$

$$\boldsymbol{e}^{(a_0+a_1x_1+a_2x_2+\ \ldots+anxn)}\Big/_{\left(\mathbf{1}+\boldsymbol{e}^{(a_0+a_1x_1+a_2x_2+\ \ldots+anxn)}\right)}$$

essa **função** é **conhecida como** função **logística** e tem o gráfico apresentado a seguir.

Aprendizado Supervisionado

Algoritmo – Regressão Logística



Aprendizado Supervisionado

Regressão Logística

- A Regressão Logística **é mais conhecida por ser usada em classificação binária**, ou seja, quando há duas classes (ex: sim/não, 0/1, positivo/negativo). Mas ela não é limitada apenas a isso.

- Ela também pode ser usada em

1) Classificação Multiclasse (mais de duas classes) usando:

1. **One-vs-Rest (OvR) ou One-vs-All**
2. **Softmax Regression** (também chamada de Regressão Logística Multinomial)

2) Classificação Multirótulo (multi-label)

Menos comum com regressão logística pura, mas possível ao adaptar o modelo para prever múltiplos rótulos binários de forma independente.

Aprendizado Supervisionado

Regressão Logística

Multiclasse:

- “Qual é o gênero principal?” => só 1 (Drama OU Comédia)

- classificar frase como: premissa OU conclusão OU outro

Python

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(multi_class="multinomial", solver="lbfgs")
model.fit(X, y)  # y é vetor (n,)
```

Aprendizado Supervisionado

Regressão Logística

Multirrótulo:

- “Quais gêneros estão presentes?” => vários (Drama E Romance E Ação) Multiclasse:
- frase pode ser: premissa + emocional + irônica ou conter múltiplas entidades

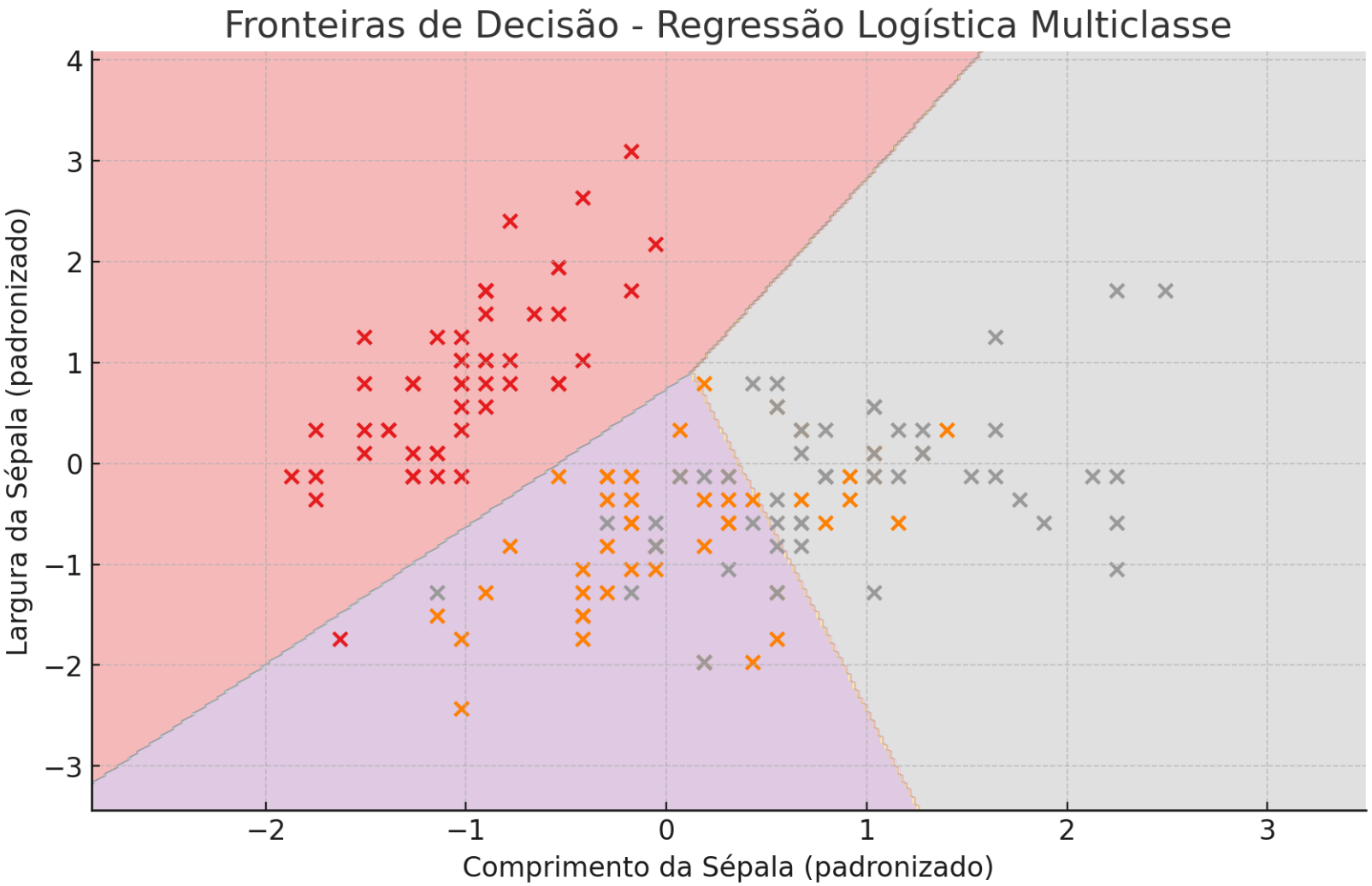
Python

```
from sklearn.multiclass import OneVsRestClassifier
from sklearn.linear_model import LogisticRegression

model = OneVsRestClassifier(LogisticRegression())
model.fit(X, y)  # y é matriz (n, k)
```

Aprendizado Supervisionado

Regressão Logística Multinomial – Exemplo: Classificar flores em setosa, versicolor ou virginica



Aprendizado Supervisionado

Algoritmos de Classificação – **Classificador k-NN**

- O método dos **k-vizinhos mais próximos** (*k-nearest neighbors* – k-NN) é um dos classificadores não paramétricos **baseado**s **em distância** mais simples e conhecidos da literatura.

Aprendizado Supervisionado

Algoritmos de Classificação – **Classificador k-NN - Funcionamento**

- **Dado** um objeto $\mathbf{x_0}$ **cuja classe se deseja inferir**
- **Encontram-se os k objetos** x_i, i = 1, ..., k da base que estejam **mais próximos a** $\mathbf{x_0}$, e, depois,
- Se **classifica o objeto** $\mathbf{x_0}$ como pertencente **à classe da maioria dos k vizinhos**.
- **Empates são decididos aleatoriamente**.
- Quando os **objetos** da base são **numéricos**, a **medida mais comum é a Euclidiana**, mas deve-se considerar o contexto da aplicação e as características da base.

Aprendizado Supervisionado

Algoritmos de Classificação – Classificador k-NN

- Apesar da simplicidade, o método k-NN **apresenta bons resultados em diversos cenários** e normalmente se comporta bem quando cada classe possui diversos objetos e a superfície de decisão é irregular.
- Esse **método é baseado em instâncias** (*instance based learning*), pois ele **determina a classe de um objeto com base na classe de outros** objetos (instâncias).
- Também **chamado de algoritmo preguiçoso** (*lazy learning*), pois **não é treinado a priori**, uma vez que **a saída é calculada apenas quando se deseja saber a classe de algum novo objeto**.

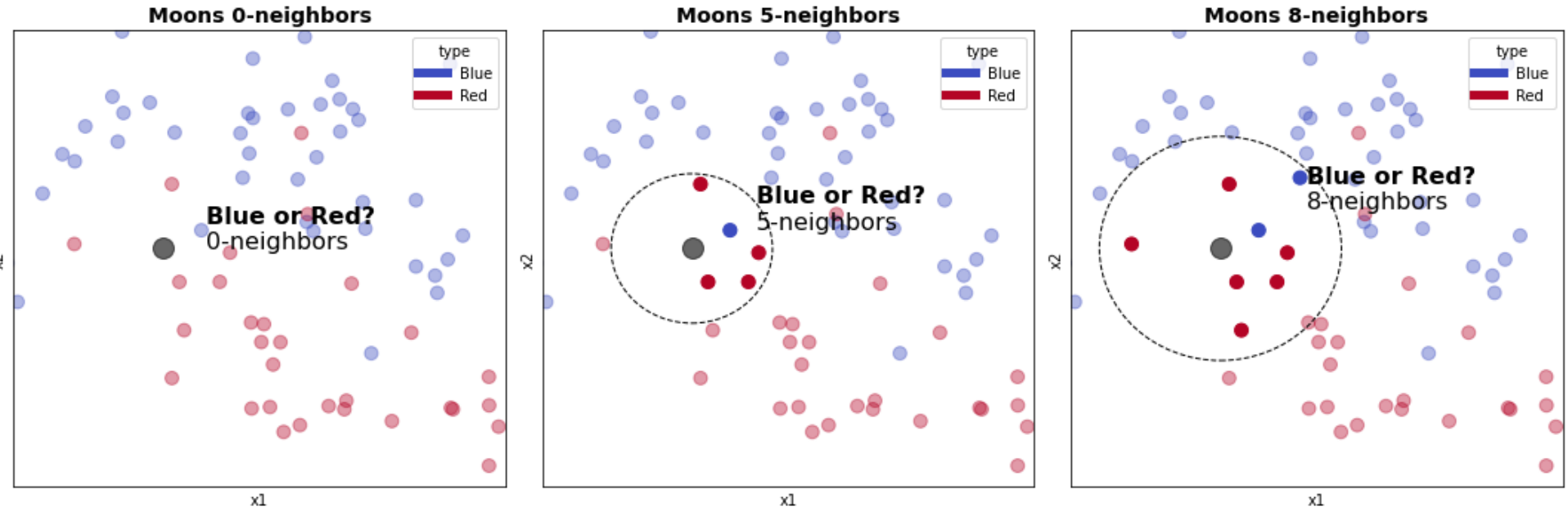
Aprendizado Supervisionado

Algoritmos de Classificação – Classificador k-NN

- Um **parâmetro a ser definido no k-NN** é o **valor de k**, ou seja, **o número de vizinhos mais próximos** a serem considerados para se definir a classe de x_0.
- **Grandes valores de k reduzem o efeito dos ruídos na classificação, mas tornam fronteiras de classe menos definida**.
- **O valor de k é definido por alguma heurística ou por tentativa e erro**.
- **O número de vizinhos pouco impacto no seu desempenho**, pois **é necessário determinar a similaridade do objeto de entrada a todos os objetos do modelo, independentemente do número de vizinhos**.

Aprendizado Supervisionado

Algoritmos de Classificação – Classificador k-NN



Aprendizado Supervisionado

Algoritmos de Classificação – Classificador k-NN – Métricas de Distância

- Para **medir a distância** entre vetores característicos, os elementos mais próximos, foi empregada **a distância mais comum, distância Euclidiana**.
- Algumas funções distância comuns:

Distância Euclidiana

$$\|ab\|_2 = \sqrt{\sum_i (a_i - b_i)^2}$$

Distância Euclidiana Quadrática

$$\|ab\|_2^2 = \sum_i (a_i - b_i)^2$$

Distância de Manhattan

$$\|ab\|_1 = \sum_i |a_i - b_i|$$

Distância Máxima

$$\|ab\|_\infty = \max_i |a_i - b_i|$$

Distância Minkowski

$$\|ab\|_{Minkowski} = (\sum_i |a_i - b_i|^p)$$

Aprendizado Supervisionado

Algoritmos de Classificação – Classificador k-NN – Métricas de Distância - Distância Cosseno

- A **Distância Cosseno para tratamento de textos em que os textos têm uma representação vetorial baseada na frequência dos termos (ou palavras)** como as representações *TF-IDF*.
- Ela **mede o ângulo formado pelos vetores** e a **distância de dois vetores a e b pode ser obtida a partir da expressão**:

$$ab_t = \|a\| \|b\| \cos(\theta)$$

- Definimos a similaridade de dois vetores como:

$$similarity(a,b) = cos(\theta) = \frac{ab^t}{\|a\| \|b\|} = \frac{\sum_{i=1}^{n} a_i b_i}{\sqrt{\sum_{i=1}^{n} a_i^2} \sqrt{\sum_{i=1}^{n} b_i^2}}$$

- E a distância: distancia(a,b)= 1– similarity(a,b)

Aprendizado Supervisionado

Algoritmos de Classificação – Classificador k-NN – Seleção de Hiperparâmetros

- Qual o melhor valor de k e a função de distância?
- **A escolha de melhores hiperparâmetros é em geral por experimentação**.
- **A ideia, então, é criarmos os diferentes modelos e avaliarmos o desempenho** (as métricas que você aprendeu aqui!) de cada um **para obtermos os melhores hiperparâmetros**.

Aprendizado Supervisionado

Algoritmos de Classificação – Classificador k-NN - Normalização

- O **cálculo de distâncias como medida de similaridade** (menor distância indicando maior similaridade) **pode**, entretanto, **apresentar grandes desvios quando empregamos variáveis com escalas muito diferentes** ou **variáveis numéricas e categóricas em conjunto**.

- **Vários modelos de aprendizado de máquina são baseados em distância como medida de similaridade** e são, portanto, **sensíveis à normalização dos dados** e **devemos aplicá-la quando empregados dados em diferentes escalas**.

Avaliação de Modelos Preditivos

Métricas de Erro – Classificação - Taxa de erro

- **A taxa de erro de f** é obtida pela comparação da classe conhecida de x_i, y_i, com a classe predita, $f(x_i)$, como abaixo:

$$err(f) = \frac{1}{n}\sum_{i=1}^{n} I(y_i \neq f(x_i))$$

- Onde ***I***(a) = 1 se a é verdadeiro e é 0 caso contrário.
- Isso indica que a taxa de erro varia entre 0 e 1.
- Valores próximos de 0 são melhores.

Avaliação de Modelos Preditivos

Métricas de Erro – Classificação - Acurácia

- **A taxa de acerto de f ou acurácia** é obtida pelo complemento da taxa de erro, ou seja:

$$ac(f) = 1 - err(f)$$

- Essa taxa também varia entre 0 e 1.
- Valores próximos de 1 são melhores.

Avaliação de Modelos Preditivos

Métricas de Erro – Classificação – Acurácia e outras Métricas

- Além da acurácia, outras possíveis métricas são: **Precisão**, **Revocação**, **F1** e a **Matriz de Confusão**.
- Essas métricas são bastante importantes e são empregadas para:
  - comparar e selecionar melhores modelos entre os diferentes modelos que podemos implementar;
  - o ajuste de parâmetros dos modelos (e que chamaremos de hiperparâmetros).

Avaliação de Modelos Preditivos

Métricas de Erro – Classificação – Matriz de Confusão

- A **matriz de confusão ilustra** o **número de predições corretas e incorretas em cada classe**.

- Para um determinado conjunto de dados, **as linhas** dessa matriz **representam as classes verdadeiras**, e **as colunas**, **as classes preditas** pelo classificador.

Avaliação de Modelos Preditivos

Problemas de duas classes – matriz de confusão

- Usualmente, **uma classe** é denotada **positiva (+)** e a outra **negativa (-).**
- Tem-se a **matriz de confusão**:

		Classe Predita	
		+	-
Classe verdadeira	+	VP	FN
	-	FP	VN

Avaliação de Modelos Preditivos

Problemas de duas classes – matriz de confusão - utilidade

- Considere a matriz de confusão abaixo obtida para avaliação de um nódulo da mama:

		Classe Predita	
		Benigno (+)	Maligno (-)
Classe original	Benigno (+)	900	0
	Maligno (-)	91	9

- O resultado para a acurácia é de 0.909.
- Não parece ruim, mas será que ele é bom?

Avaliação de Modelos Preditivos

Problemas de duas classes – Matriz de Confusão - Exemplo

- Dada uma **base de dados** contendo **1000 e-mails** para classificação entre spam e normal, a **aplicação de um algoritmo de classificação** resultou na **matriz de confusão** abaixo:

		Classe Predita	
		Spam	Normal
Classe original	Spam	100	90
	normal	10	800

A partir dela, obtém-se os valores:

VP = 100, VN = 800

FN = 90, FP = 10.

n = VP + VN + FP + FN = 100 + 800 + 10 + 90 = 1000

Avaliação de Modelos Preditivos

Problemas de duas classes – Medidas de Desempenho

- **A partir da matriz de confusão**, uma série de outras medidas pode ser derivada, algumas delas são:
  - **Taxa de acerto ou acurácia total**
  - **Precisão (classe positiva/classe negativa)**
  - **Sensibilidade ou Revocação ou *recall* (TVP/TVN)**

Avaliação de Modelos Preditivos

Problemas de duas classes – Medidas de Desempenho – Taxa de acerto ou acurácia total

- **Taxa de acerto ou acurácia total**:
  - **Número de previsões corretas**.
  - É **útil** apenas quando existe o mesmo número de observações (amostras) em cada classe, ou seja, **classes balanceadas**.
  - Usando esta métrica, **quando não há o mesmo número de observações para cada classe** (categoria) é **adequado fazer o balanceamento**, ou seja, deixar todas as classes com a mesma quantidade de objetos (observações).

Avaliação de Modelos Preditivos

Problemas de duas classes – Medidas de Desempenho – Taxa de acerto ou acurácia total

- **Taxa de acerto ou acurácia total**: calculada pela soma dos valores da diagonal principal da matriz, dividida pela soma dos valores de todos os elementos da matriz.

$$Accuracy = \frac{TotaldeAcertos}{TotaldeCasos}$$

		Classe Predita	
		+	-
Classe verdadeira	+	VP	FN
	-	FP	VN

$$ac(f) = \frac{VP + VN}{n}$$

Avaliação de Modelos Preditivos

Problemas de duas classes – Medidas de Desempenho – Taxa de acerto ou acurácia total - Exemplo

- Considerando o exemplo dos 1000 e-mails onde a classe positiva equivale ao e-mail ser um SPAM. Além dos valores da matriz de confusão: VP = 100, VN = 800, FN = 90, FP = 10.

$$ac(f) = \frac{VP + VN}{n} = \frac{100 + 800}{1000} = \frac{900}{1000} = 0{,}9$$

		Classe Predita	
		Spam	Normal
Classe original	Spam	100	90
	normal	10	800

Avaliação de Modelos Preditivos

Matriz de Confusão com n categorias para a Classe

- Apesar de as bases de dados binárias serem muito comuns, elas são um caso particular das bases com múltiplas classes.
- Em um **problema de predição multiclasse**, a **matriz de confusão** possui **nas linhas** as **classes originais** e nas **colunas**, as **classes preditas**, sendo uma linha e uma coluna para cada classe.

Avaliação de Modelos Preditivos

Matriz de Confusão com n categorias para a Classe

- Matriz de Confusão Mc

		Classe Predita			
		Classe 1	Classe 2	...	Classe n
Classe Original	Classe 1	C_{11}	C_{12}	...	C_{1n}
	Classe 2	C_{21}	C_{22}	...	C_{2n}
	...	...	...	...	...
	Classe n	C_{n1}	C_{n2}	...	C_{nn}

- Elementos da Matriz de Confusão (C_{ij}) indicam a quantidade de objetos da classe i que foram classificados como pertencentes à classe j.

Avaliação de Modelos Preditivos

Matriz de Confusão com n categorias para a Classe

- **Nem todas** as **medidas de avaliação** aplicadas aos problemas de classificação **binária podem ser utilizadas** em **problemas de múltiplas classes**.
- Por exemplo, a **equação de acurácia** deve ser adaptada ao problema e calculada como:

$$ac(f) = \frac{\sum_{i=1}^{n} C_{ii}}{\sum_{i=1}^{n} \sum_{j=1}^{n} C_{ij}}$$

Avaliação de Modelos Preditivos

Matriz de Confusão com n categorias para a Classe – Exemplo

- **Matriz de confusão** resultante de **1000 condições de treinamento de navegação de robôs**:

		Classe Predita			
		Norte	Sul	Leste	Oeste
Classe Original	Norte	215	8	10	12
	Sul	8	220	9	14
	Leste	13	10	217	12
	Oeste	10	15	14	213

$$ac(f) = \frac{215 + 220 + 217 + 213}{1000} = \frac{865}{1000} = 0{,}865$$

Ou seja, a **acurácia é 86,5%.**

Aprendizado Supervisionado

Passos para geração de um modelo preditivo - Python

- Vamos entender como empregar um estimador

compreendendo cada passo:

1. Realizar o **import** dos pacotes necessários e do(s) algoritmo(s)
2. Carregar o *dataset* **anotado** em um **Dataframe**
3. A partir do **Dataframe**, preparar os dados, definir as entradas e saídas do modelo.
4. Amostragem do *dataset*, por exemplo, divisão em treino (65% a 80%) e teste (20% a 35%). Pode depender do tamanho do *dataset.*

Aprendizado Supervisionado

Passos para geração de um modelo preditivo - Python

5. Definir a Classe de Modelos a ser empregada.
6. Treinar o Modelo. Isto é, buscar os parâmetros que melhor ajusta o modelo aos dados.
7. Avaliar o modelo por meio de métricas de avaliação de erro.
8. Prever novos casos. Aplicamos o modelo para novos casos.

Obs.: Esses passos aplicam-se a a qualquer algoritmo supervisionado

Aprendizado Supervisionado

1. Import dos pacotes - Python

- Antes de mais nada devemos importar os pacotes necessários para a execução do nosso modelo (numpy, pandas, matplotlib, seaborn, ...).
- Por exemplo:

```
# Importação dos módulos necessários
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Aprendizado Supervisionado

1. Import dos pacotes - Python

- Cada algoritmo de aprendizado na API do scikit-learn é exposto como objeto chamado “estimador”.
- Por exemplo:
  - Regressão Linear

    from sklearn.linear_model import **LinearRegression**

  - **Regressão Logística**

    from sklearn.linear_model import **LogisticRegression**

Aprendizado Supervisionado

2. Carregar dataset em um dataframe - Python

- Considere um *dataset*, por exemplo, o dataset "**breast-cancer-wisconsin.csv**"* que tem **dados a respeito do diagnóstico de tumores** empregando algumas das suas **33** características (**atributos**) : **diagnosis**, **radius_mean**, **texture_mean**, **perimeter_mean**, **smoothness_mean**, **compactness_mean**, **concavity_mean**, **concave**, **points_mean**, **symmetry_mean**, **fractal_dimension_mean**, **area_mean** etc.
- A sua **distribuição de classes** é : 357 **benignos**, 212 **malignos.**

df_cancer = pd.read_csv('breast-cancer-wisconsin.csv')

Fonte: https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+%28Diagnostic%29. Data da Consulta: 14/09/2025.

Aprendizado Supervisionado

3. A partir do Dataframe, preparar os dados, definir as entradas e saídas do modelo - Python

- Essa **etapa** pode **requerer uma série de transformações e seleções dos dados** e, normalmente, é uma **parte** bastante **custosa da implementação completa** dos modelos.
- Por exemplo:
  - Considerando os dados preparados e **utilizando somente** as características: **diagnosis, radius_mean, texture_mean e perimeter_mean**.
  - No qual **diagnosis** é o **atributo alvo (saída)**.

Aprendizado Supervisionado

3. A partir do Dataframe, preparar os dados, definir as entradas e saídas do modelo - Python

```
df_cancer = df_cancer[['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean']]
df_cancer.head()
```

	diagnosis	radius_mean	texture_mean	perimeter_mean
0	M	17.99	10.38	122.80
1	M	20.57	17.77	132.90
2	M	19.69	21.25	130.00
3	M	11.42	20.38	77.58
4	M	20.29	14.34	135.10

Aprendizado Supervisionado

3. A partir do Dataframe, preparar os dados, definir as entradas e saídas do modelo - Python

```
X = df_cancer[['radius_mean', 'texture_mean', 'perimeter_mean']]
X
```

	radius_mean	texture_mean	perimeter_mean
0	17.99	10.38	122.80
1	20.57	17.77	132.90
2	19.69	21.25	130.00
3	11.42	20.38	77.58
4	20.29	14.34	135.10
...	...	...	...
564	21.56	22.39	142.00
565	20.13	28.25	131.20
566	16.60	28.08	108.30
567	20.60	29.33	140.10
568	7.76	24.54	47.92

569 rows × 3 columns

Aprendizado Supervisionado

3. A partir do Dataframe, preparar os dados, definir as entradas e saídas do modelo - Python

```
y = df_cancer['diagnosis']
y
```

```
0      M
1      M
2      M
3      M
4      M
      ..
564    M
565    M
566    M
567    M
568    B
Name: diagnosis, Length: 569, dtype: object
```

Aprendizado Supervisionado

4. Amostragem do conjunto de dados em treino e teste (20% a 35%) - Python

- Algumas estratégias podem ser utilizadas para a amostragem dos dados, na tentative de diminuir o risco de *overfitting*.
- Uma estratégia é a criação de Conjuntos de dados de Treinamento e Teste.
- Dependendo do tamanho do dataset, pode ser utilizado entre 20% e 35% no tamanho do conjunto de teste.
- Necessário um **import** para a amostragem do modelo como segue:

```
from sklearn.model_selection import train_test_split
```

- E fazer o uso da função **train_test_split()**.

Aprendizado Supervisionado

4. Amostragem do conjunto de dados em treino e teste (20% a 35%) - Python

X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.3, random_state=123)

Onde:

- **X_train, y_train** será a parte dos dados empregada para o treinamento dos parâmetros do modelo
- **X_test, y_test** será a parte dos dados empregada para avaliar o modelo preditivo ajustado.
- **X** é o conjunto de atributos de entrada do *dataset*
- **y** é o conjunto de atributos de saída do *dataset*
- **stratify = y** indica que será utilizada a mesma proporção de dados de cada uma das classes para o conjunto de dados de treino e teste, tendo por base a proporção no conjunto original.
- **test_size**: proporção dos dados de teste, no caso, 0.3 = 30%.
- **random_state**: número utilzado para permitir a reprodução da divisão realizada em outros testes.

Aprendizado Supervisionado

5. Definir a Classe de Modelos a ser empregada - Python

- Instanciando o modelo com seus parâmetros.

- Por exemplo:

model = LogisticRegression(max_iter=1000)

- Parâmetros: **max_iter** valor padrão é 100.
  - **max-iter**: Número máximo de iterações necessárias para a convergência do modelo.

Fonte: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html. Data da consulta: 14/09/2025.

Aprendizado Supervisionado

5. Definir a Classe de Modelos a ser empregada

- O **modelo vai fornecer uma estimativa de probabilidade de diagnosis benigno** com base nos valores de radius_mean, texture_mean e perimeter_mean.

$$y_{\text{pred}} = P(\text{diagnosis} = B) = \frac{exp(a_0 + a_1\text{radius_mean} + a_2\text{texture_mean} + a_3\text{perimeter_mean})}{1 + exp(a_0 + a_1\text{radius_mean} + a_2\text{texture_mean} + a_3\text{perimeter_mean})}$$

- **Os valores dos atributos de entrada precisam ser numéricos.**

Aprendizado Supervisionado

6. Treinar o modelo

- Nesta etapa, o modelo é treinado, buscando-se os parâmetros que melhor ajustam modelo aos dados.

```
# realiza o treinamento do modelo
model.fit(X_train,y_train)

print( model.coef_, model.intercept_)
```

- Os valores coef_ e intercept_ são os parâmetros estimados para regressão logística e varia dependendo da quantidade de atributos.

Aprendizado Supervisionado

7. Avaliar o modelo

A partir do treinamento do modelo, é possível avaliar o seu desempenho por meio de métricas de avaliação de erro.

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y, y_pred)
print('\nScore de Acuracidade (modo 1):\n')
print(accuracy)
```

Aprendizado Supervisionado

7. Avaliar o modelo – Acurácia do Modelo - Python

- **Acuracidade do model**o **sobre o nosso conjunto de treinamento X**, isto é, o **percentual de acerto do nosso modelo quando comparamos com os rótulos** (as respostas) **presentes no nosso conjunto**.
- Você pode obter o mesmo valor apenas contando o **número de acertos da predição sobre o total de casos**, ou seja,

sum(model.predict(X) == y) / len(X)

Aprendizado Supervisionado

8. Prever novos casos - Python

- O método **predict()** fornece então a **predição de dados com base nos parâmetros estimados do modelo**.

```
# faz a previsão nos dados de teste
y_pred = model.predict(X_test)

# Apresenta a quantidade de valores incorretos previstos
sum(y_test != y_pred)
```

Aprendizado Supervisionado

8. Prever novos casos - Python

- Ainda existem **outros preditores úteis que retornam a probabilidade ou o log da probabilidade de cada classe**, o que é **útil quando se deseja refinar os resultados** ou ainda desejamos **empregar esses valores para solução de problemas multiclasse**.

- **model. predict_proba(X)[0]**

- **model. predict_log_proba(X)[0]**

Bibliografia

BÁSICA:

- AGGARWAL, Charu C. **Artificial Intelligence: A Textbook**. New York: Springer: 2021.
- CHOLLET, François. **Deep Learning with Python, 2ed.** Shelter Island: Manning, 2021.
- GÉRON, Aurélien. **Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems**, 2 ed**.** Sebastopol: O'Reilly, 2019.

COMPLEMENTAR:

- GOODFELLOW, Ian; BENGIO, Yoshua, COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016.
- RASCHKA, Sebastian; MIRJALILI, Vahid. **Python Machine Learning**. 3 ed. Birmingham: Packt, 2017.
- RUSSEL, Stuart; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 3 ed. Upper Saddle River: Pearson, 2010.
- TAN, Pang-Ning; STEINBACH, Michael; KUMAR, Vipin. **Introduction to Data Mining**. 2 ed. Upper Saddle River: Pearson, 2018.
- VANDERPLAS, Jake. **Python Data Science Handbook**. Sebastopol: O'Reilly, 2017.

ADICIONAIS:

- FACELI, Katti et al. **Inteligência artificial: uma abordagem de aprendizado de máquina**. 2ª Ed. Rio de Janeiro: LTC- Livros Técnicos e Científicos, 2021.
- LUGER, George F. **Inteligência Artificial** - 6ª ed. São Paulo: Pearson Education do Brasil, 2015.
- RUSSELL, Stuart; NORVIG, Peter. **Inteligência artificial: Uma Abordagem Moderna** - 4ª. Ed. GEN LTC, 2022.